GraphMemShield: Auditing and Mitigating Cross-Session Privacy Leakage in Dynamic Knowledge-Graph Memories for Graph-Backed Applications

Anonymous Author(s)

***Abstract*—Persistent knowledge-graph (KG) memories are becoming a practical substrate for graph-backed assistants: they extract entities, relations, timestamps, and provenance from one session and later retrieve k-hop neighborhoods to ground another response. This creates a privacy boundary that is neither standard model memorization nor ordinary document-RAG leakage: the sensitive object is a live, multi-user, provenance-tagged graph. We introduce GraphMemShield, a reproducible audit and mitigation framework for this setting. It formalizes dynamic KG memory under edge-event adjacency, implements attacks for cross-session edge exposure, session linkage, temporal inference, adaptive probing, and response leakage, and evaluates defenses based on provenance-aware bounded sharing and fixed-universe randomized response. Across five workloads—synthetic, Docker-backed, enterprise/health/finance, MultiWOZ 2.1, and Enron-style communication graphs—unguarded retrieval exposes victim-owned edges and sensitive response content. On the 2,304-edge enterprise benchmark, multi-hop retrieval increases unguarded leakage from 9 to 12 victim edges; a full backend–retrieval–response pipeline leaks 12 victim edges and 8 sensitive terms without a guard. Bounded sharing makes the privacy–utility tradeoff explicit: increasing the per-pair budget from 0 to 10 raises non-sensitive utility retention from 0.0 to 0.614 while keeping unique victim leakage capped at three edges. We further show that this guarantee depends on correct sensitivity provenance, add lexical and semantic black-box response leakage scoring, and separate a non-DP edge-suppression heuristic from a fixed-universe randomized-response mechanism with explicit composition and full-graph-release scope accounting.**

***Index Terms*—Knowledge-graph memory, cross-session leakage, provenance, graph-augmented retrieval, privacy auditing, session linkage, bounded sharing.**

I. Introduction

GRAPH-BACKED applications now sit at the intersection of conversational interfaces, graph-augmented retrieval, and persistent memory services. A single deployment may parse user utterances into $(\textit{subject}, \textit{relation}, \textit{object})$ triples, store them in a property graph or KG index, and later retrieve k-hop neighbourhoods to ground responses for the *same* or for a *different* user. The resulting backend is no longer a passive read-only knowledge base: it is a continuously evolving multi-tenant graph whose retrieval behaviour is shaped by every prior session.

Manuscript prepared for IEEE Transactions on Information Forensics and Security. The associated open-source artifact is available at `https://github.com/SonHaXuan/GraphMemShield`.

Recent privacy research has established that text-level memory can leak verbatim facts [1], [2], that retrieval-augmented systems expose the documents in their indices [3], [4], and that learned graph models suffer from link-stealing and embedding-inversion attacks [6]–[8]. Yet the deployment pattern that combines all three—*a dynamic, multi-user KG memory consulted by a black- or gray-box retrieval service*—has received limited focused attention. In particular, no existing benchmark we are aware of jointly measures (i) cross-session edge exposure, (ii) anonymized-session linkage by graph structure, and (iii) write-order inference from observable provenance metadata, all on the *same* graph, with the *same* defensive policy applied at retrieval time.

Problem. A user u_v confides a sensitive relation—*visited a cardiology clinic*—to a graph-memory service. A different user u_a later asks an unrelated question whose 1-hop graph expansion reaches the same entity nodes. Without a provenance-aware policy, the service returns edges *owned by* u_v, and the textual response reveals or strongly hints at the sensitive fact. Even if responses are sanitised, the observable *retrieved subgraph*, its *insertion order*, and its *degree signature* can be used to: (a) re-identify u_v's sessions across pseudonymised IDs and (b) reconstruct the sequence of writes that introduced the sensitive neighbourhood.

Main findings. Graph memory leakage is not merely a retrieval-count artifact. On the enterprise benchmark, moving from one-hop to two-hop expansion saturates the graph and raises unguarded victim leakage from 9 to 12 edges; the hybrid backend–retrieval– response pipeline then carries the same leakage into final text. On MultiWOZ 2.1, the public-corpus run produces a 36,408-edge graph and 67 unguarded victim-edge leaks. On an Enron-style communication graph, the same audit surface appears in sender–recipient–subject structure. Across these settings, strict isolation removes cross-session exposure, but the more operational result is bounded sharing: it exposes a tunable utility/risk curve and remains bounded under multi-hop expansion, provided sensitivity provenance is correct.

Contributions. This paper makes the following contributions:

- We formalise dynamic, multi-session KG memory with provenance-aware edges and define three concrete privacy failure modes: cross-session retrieval contamination, session-linkage leakage, and temporal-path exposure.

- We instantiate three reproducible baseline attacks—CROSSSESSIONPROBE, SESSIONGRAPHLINK, and TEMPORALPATHINFER—and extend them with adaptive probing, multi-hop retrieval, black-box response scoring, and semantic leakage scoring.
- We present GRAPHMEMGUARD, a retrieval-time defense combining provenance filtering, blocked-sensitivity sets, and a bounded per-pair exposure budget, together with RANDOMIZEDEDGEADMISSION, a seeded write-time edge-suppression heuristic, and a fixed-universe randomized-response release mechanism for formal per-edge privacy over a public candidate edge universe.
- We release an open-source artifact (Python package, unit tests, data fixtures, and generated outputs) that reproduces results on synthetic, Docker-backed, enterprise-style, MultiWOZ, and Enron-style communication workloads.
- We include a persistent property-graph backend experiment, Kuzu latency measurements, 1-WL, graph-edit, and embedding session-linkage baselines, timestamp-hidden temporal inference, provenance-error robustness, and full-graph privacy scope accounting.

Roadmap. Section II surveys prior work on memory leakage, retrieval-augmented privacy, and graph privacy. Section III defines the system and threat models. Section IV presents the GraphMemShield abstractions, attacks, and defenses. Section V describes the open-source implementation and datasets. Section VI reports the attack, defense, backend, public-corpus, and privacy-accounting results. Section VII discusses scope, limitations, and ethics. Section VIII concludes.

II. RELATED WORK

Memory and prompt leakage. Carlini *et al.* [1] demonstrated that large language models memorise and emit training data verbatim under crafted prompts. Subsequent work expanded this to fine-tuned conversational systems [2]. These attacks treat the model itself as the leaking surface. Our work assumes that the model is benign and focuses on the *external memory store* that mediates retrieval across sessions.

Retrieval-augmented generation (RAG) privacy. Zeng *et al.* [3] formalised leakage from RAG document indices, and Qi *et al.* [4] demonstrated chunk extraction by adaptive querying. Graph-augmented RAG has its own extraction surface; recent work [5] extracted entities and relations from graph-backed RAG by exploiting the k-hop expansion. GraphMemShield differs by treating the KG as a *multi-user, multi-session, append-only* structure and by isolating the cross-session boundary as the primary privacy artifact.

Graph privacy. Link-stealing [6], graph model inversion [7], and embedding-leakage attacks [9] target trained GNNs and learned representations. Differentially private graph release mechanisms [10]–[12] and DP graph neural networks [13], [14] provide formal guarantees on *static* graphs or *model parameters*, not on the live retrieval pipeline of an evolving multi-user memory. Our framework is intentionally aligned with these formalisms (we adopt edge-event adjacency), but the formal claim is deliberately narrow: only fixed-universe candidate-edge bits receive a DP statement, and full graph publication requires every released edge bit to enter that universe.

Membership inference and re-identification. Membership inference attacks [15], [16] reveal whether a record participated in training. Graph-level re-identification [17] has shown that behavioural traces such as walks or interaction histories serve as fingerprints. SESSIONGRAPHLINK adapts this insight to *retrieval-time graph fingerprints*, ranking anonymised sessions by structural similarity rather than by raw entity overlap.

Provenance-aware access control. Provenance graphs are widely used in audit and compliance pipelines [18]. Our GRAPHMEMGUARD is a lightweight realisation of provenance-aware retrieval: edges carry their owning session and sensitivity label, and the policy decides admission *before* the query result enters the response context. We evaluate it both as a strict isolation baseline (b=0) and as a budgeted exposure mechanism (b>0).

III. SYSTEM AND THREAT MODEL

A. Dynamic KG Memory

We model a graph memory as $\mathcal{G}_t = (V_t, E_t, \pi)$ at logical time t, where every edge $e \in E_t$ carries provenance $\pi(e) = (\textit{owner}, \textit{user}, \textit{turn}, \textit{sensitivity}, \textit{ts})$. An edge is added when a session s writes a relation extracted from a turn *turn*. The service exposes a retrieval primitive $\textsc{Retrieve}(q, s_r, k) \to R$ that returns the k-hop expansion around query q from the perspective of requester session s_r. Visibility is governed by a guard Γ such that each edge is admitted only if $\Gamma(e, s_r) = \text{True}$.

B. Adjacency

For privacy analysis we adopt *edge-event adjacency*: two histories $\mathcal{G}, \mathcal{G}'$ are neighbours if they differ in exactly one provenance-tagged edge e. This matches the granularity at which our write-time admission mechanism operates and admits a future composition argument over t writes.

C. Adversaries

We consider three adversaries:

1) **Black-box requester** ($\mathcal{A}_b$). Issues queries and observes only final responses.
2) **Gray-box requester** ($\mathcal{A}_g$). Additionally observes the retrieved subgraph (a common setting for transparency or developer modes).
3) **Honest-but-curious operator** ($\mathcal{A}_o$). Inspects metadata for ground-truth evaluation; never used as an attacker against a deployed system, only to audit.

Our experiments instantiate $\mathcal{A}_g$ to measure the retrieval surface directly and $\mathcal{A}_b$ to score whether final response text leaks victim-owned edges or sensitive terms. The gray-box setting gives an upper bound on retrieval contamination, while the black-box setting captures what an ordinary requester may actually observe.

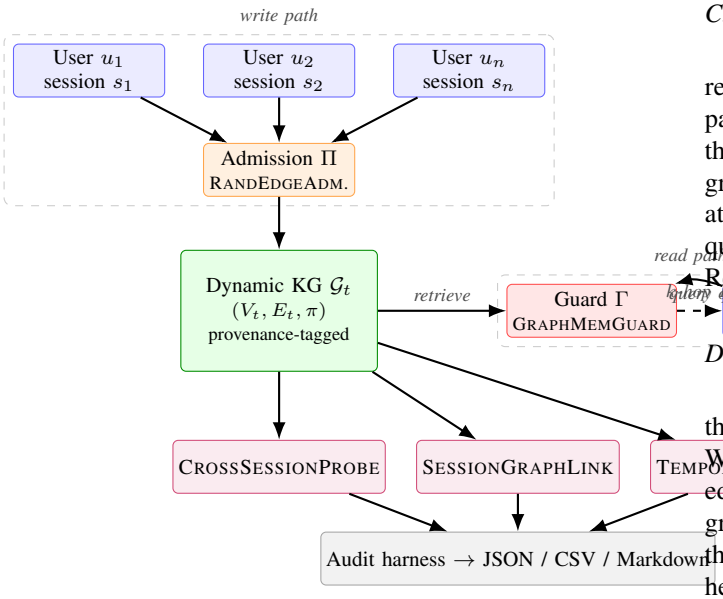

Fig. 1. System architecture of GraphMemShield. Write-time admission (Π) and read-time guard (Γ) bracket the dynamic KG; the audit harness exposes attack and metric modules.

D. Sensitive Objects

The sensitive objects are: (i) edge existence $\mathbf{1}[e \in E_t]$, (ii) edge ownership $\mathit{owner}(e) = s_v$, (iii) the local ego-graph of a victim session, and (iv) the temporal sequence of writes producing a sensitive neighbourhood.

E. Out-of-Scope

We do *not* model attacks against the underlying storage layer, side channels of the retrieval service, or model-internal memorisation. We assume responses are produced by a downstream component and focus on the graph retrieval boundary.

IV. The GraphMemShield Framework

A. Overview

Figure 1 shows the framework. Sessions write into the dynamic KG via an admission policy Π. Retrieval consults the KG through a guard Γ. Attack and scoring modules consume the audit interface, including cross-session probing, session linkage, temporal ordering, adaptive multi-hop probing, and response-level leakage scoring. The evaluation harness aggregates JSON, CSV, and Markdown reports.

B. Attack 1: CrossSessionProbe

Given an attacker session s_a and a victim session s_v, the probe issues a budgeted set of queries Q and counts retrieved edges whose owner is s_v. We report (i) *leaked edge count* (unique victim-owned edges over Q), (ii) *leakage events* (occurrences across queries), (iii) the *leakage rate* $|L|/|U|$ where L are leaked edges and U are unique retrieved edges, (iv) the *event leakage rate*, and (v) the *per-query leak rate*.

C. Attack 2: SessionGraphLink

Each session is mapped to a multiset of features, including relation counts, sensitivity bigrams, relation co-occurrence pairs, and a degree histogram. Crucially, we incorporate the 1-Weisfeiler-Lehman (1-WL) kernel hash of local ego-graphs, providing a strong structural equivalence baseline. The attacker ranks candidate sessions by cosine similarity to a query session. Reported metrics are top-k accuracy and Mean Reciprocal Rank (MRR).

D. Attack 3: TemporalPathInfer

Given a query and a k-hop expansion, the attacker orders the returned edges by their observable timestamp metadata. We report (i) *prefix ordering accuracy* for the leading $|P|$ edges, (ii) *pairwise ordering accuracy* restricted to comparable ground-truth pairs, and (iii) edge precision/recall/F1 against the ground-truth path. We also evaluate a timestamp-hidden heuristic based on relation priority and path continuity to test whether ordering signals remain after explicit timestamps are removed.

E. Defense 1: GraphMemGuard

GraphMemGuard is parameterised by a triple $(\alpha, b, \mathcal{B})$ where $\alpha \in \{0, 1\}$ enables cross-session sharing, $b \in \mathbb{Z}_{\geq 0}$ caps the unique cross-session edges admitted per requester–owner pair, and $\mathcal{B}$ is the set of blocked sensitivity labels. The predicate is

$$\Gamma(e, s_r) = \begin{cases} \text{True} & \mathit{owner}(e) = s_r \\ \text{False} & \mathit{sens}(e) \in \mathcal{B} \\ \text{False} & \alpha = 0 \\ \mathbf{1}[c(s_r, \mathit{owner}(e)) < b] & \text{otherwise} \end{cases} \tag{1}$$

where $c(\cdot, \cdot)$ tracks unique cross-session admissions per pair. The strict configuration $\alpha = 0$, $b = 0$ realises full provenance/session isolation and yields a hard upper bound on the defense.

Theorem 1: For any requester session s_r and owner session s_o, a GraphMemGuard policy with cross-session budget b admits at most b cross-session exposure events from s_o to s_r. If a sensitivity label $\ell \in \mathcal{B}$ is correctly assigned, then edges with label ℓ have zero cross-session exposure.

Proof. For owner-equal edges the predicate returns true and no cross-session exposure is counted. For owner-different edges, the predicate first rejects every edge whose sensitivity lies in $\mathcal{B}$. If the edge is not blocked, it is admitted only while $c(s_r, s_o) < b$, and each admitted cross-session edge increments the same counter through Record. Thus no more than b admitted cross-session exposure events can be recorded for the pair. Correctly labelled blocked edges are rejected before the budget check, so their cross-session exposure is zero. ■

F. Defense 2: RandomizedEdgeAdmission

This write-time module accepts a sensitive edge with probability $p = \sigma(\varepsilon) = \frac{e^{\varepsilon}}{1+e^{\varepsilon}}$ and suppresses it with probability

$1-p$. Non-sensitive edges are always admitted. We use the parameter name ε only to align the sweep with randomized-response intuition: larger values retain more sensitive edges and therefore usually preserve more utility while allowing more downstream leakage.

Non-DP status. The current mechanism is *not* ε-edge differentially private under edge-event adjacency. If neighbouring histories differ by one sensitive edge e, an output containing e has positive probability when e is present and zero probability when e is absent, so the likelihood ratio is unbounded. A formal edge-DP write-time release mechanism would need a public candidate edge universe and randomized response over both present and absent sensitive candidates, plus composition accounting over repeated writes. We therefore report RANDOMIZEDEDGEADMISSION only as a seeded suppression heuristic for stress-testing the audit harness.

G. Defense 3: Fixed-Universe Randomized Response

For formal privacy experiments, we additionally implement FIXEDUNIVERSERANDOMIZEDRESPONSE. Let $\mathcal{U}$ be a public candidate set of protected edges and let each graph history induce a bit vector $x \in \{0,1\}^{|\mathcal{U}|}$ over edge presence. For each candidate edge independently, the mechanism outputs bit 1 with probability $\frac{e^{\varepsilon}}{1+e^{\varepsilon}}$ when $x_i = 1$ and with probability $\frac{1}{1+e^{\varepsilon}}$ when $x_i = 0$. The guarantee is scoped only to protected candidate-edge membership: the universe $\mathcal{U}$, candidate endpoints, relation labels, and the number of releases are public. Non-protected edges and graph statistics released outside $\mathcal{U}$ are outside this privacy claim. Releasing the same protected universe repeatedly incurs sequential composition cost, which we report explicitly in the evaluation.

Theorem 2: FIXEDUNIVERSERANDOMIZEDRESPONSE satisfies ε-local differential privacy for each protected candidate edge bit and $k\varepsilon$ privacy under basic composition for k released protected bits.

Proof. For any output bit $y_i \in \{0,1\}$ and neighbouring inputs x_i, x_i' that differ in one candidate edge, the likelihood ratio is at most $\frac{e^{\varepsilon}/(1+e^{\varepsilon})}{1/(1+e^{\varepsilon})} = e^{\varepsilon}$; the same bound holds for $y_i = 0$. Independent release over k protected bits composes by the standard sequential composition theorem. ■

H. Algorithmic Summary

Algorithm IV-H states the guarded retrieval procedure that powers all reported numbers.

Algorithm 1. GUARDEDRETRIEVE

Input: query q, requester s_r, hops k, guard Γ
Output: retrieval result R

$S \leftarrow \{e \in E : \mathrm{match}(e, q) \wedge \Gamma(e, s_r)\}$
Initialise BFS queue with endpoints of S, depth 0
while queue non-empty and depth $< k$ **do**
  Pop (v, d)
  for each $e \in \mathrm{adj}(v)$ s.t. $\Gamma(e, s_r)$ **do**
    $S \leftarrow S \cup \{e\}$
    enqueue neighbour at $d{+}1$
  end for
end while
for $e \in S$ **do**
  record exposure Γ.RECORD(e, s_r)
end for
return $R = (q, s_r, S, \mathrm{nodes}(S))$

V. IMPLEMENTATION

A. Open-source Package

The framework is published as the Python package `graphmemshield`, organised into `core/` (graph and types), `attacks/`, `defenses/`, `datasets/`, `evaluation/`, and `adapters/`. The package is installed in editable mode for reproducible experiments and is exercised by 41 unit tests covering ingestion, retrieval, attacks, defenses, and metrics.

B. Datasets

Five workloads back the reported numbers:

Synthetic multi-session graph. A deterministically generated synthetic graph simulating 20 users with 3 sessions each (60 sessions total), yielding 134 nodes and 159 edges in the current artifact. The generator weaves motifs across four domains (*health, finance, work, location*), applying sensitivity labels such that GRAPHMEMGUARD and RANDOMIZEDEDGEADMISSION can selectively filter edges.

Docker-backed read-path graph. A de-identified dialogue benchmark with 12 records, 6 users, 12 sessions, 36 nodes, and 48 edges. Records are seeded into MongoDB and read through the Cloud API before being converted into the GraphMemShield memory graph. This evaluates the read-path integration and should not be interpreted as a full end-to-end encrypted proof pipeline.

Enterprise/health/finance benchmark. To stress the attacks beyond toy scale, we generate a deterministic de-identified benchmark with 576 dialogue records, 48 users, 192 sessions, 162 nodes, and $2,304$ provenance-tagged edges. The workload mixes health, finance, work, and location motifs and is exported as JSONL so that approved public or institutional traces can be substituted using the same schema.

Public-format ingestion. We add parsers for common MultiWOZ JSON exports and Enron-style maildir trees. The artifact tests these loaders on schema-faithful fixtures and maps both formats into the same `DialogueRecord` relation schema used by all attacks and defenses. We run MultiWOZ 2.1 end-to-end by downloading the upstream public zip, capping the artifact run at $1,000$ dialogues, and producing $6,613$ records, 955 sessions, $2,708$ nodes, and $36,408$ edges.

Enron-style communication graph. We add an Enron-maildir-compatible communication-graph experiment. When `GRAPHMEMSHIELD_ENRON_MAILDIR` points to an approved local maildir, the loader ingests that corpus directly; otherwise the artifact generates a reproducible Enron-style fixture with 160 messages, 6 users, 160 subject-thread sessions, 179 nodes, and 447 edges. This workload stresses sender–

recipient–subject structure rather than task-oriented dialogue slots.

Persistent property graph. We add a SQLite-backed property-graph adapter with explicit node and edge property tables. The enterprise benchmark is written to disk, read back, and evaluated through the same attack harness, providing a persistent backend experiment while keeping the artifact self-contained. We also provide an optional Kuzu adapter; the experiment records Kuzu availability and runs the same roundtrip when the local Kuzu Python package is present.

C. Utility Metrics

To measure the privacy-utility tradeoff, we track the *utility retention rate*: the fraction of non-sensitive edges that remain reachable for cross-session queries under a given defense policy, compared to the unmitigated baseline.

D. Reproducibility

The core artifact is reproduced by:

```
python examples/run_experiments.py
python examples/run_privacyguard_batch_experiment.py
python examples/generate_enterprise_benchmark.py
python examples/run_tifs_revision_experiments.py
python examples/run_multiwoz_experiment.py
python examples/run_enron_experiment.py
python examples/generate_aggregate_report.py
```

which write `output/synthetic_experiments.{json,csv,md}`, `output/privacyguard_batch_results.{json,csv,md}`, `output/tifs_revision_results.{json,csv,md}`, `output/multiwoz_results.{json,csv,md}`, `output/enron_results.{json,csv,md}`, and the 391-row `output/aggregate_results.{json,csv,md}`.

VI. Experimental Evaluation

A. Research Questions

RQ1. How much victim-owned graph memory does a baseline graph-memory service expose to an unrelated session?
RQ2. Does provenance/session isolation eliminate the exposure, and at what budget level can controlled cross-session sharing be allowed without leaking sensitive labels?
RQ3. How effectively can structural fingerprints re-identify anonymised sessions, and what is the gain from semantic labels?
RQ4. What does observable provenance metadata reveal about write-order and sensitive paths?
RQ5. How much final-response leakage remains under black-box response scoring?
RQ6. How do stronger session-linkage and timestamp-hidden temporal baselines perform?
RQ7. How does fixed-universe randomized response trade candidate-edge privacy for utility and synthetic edge injection?
RQ8. How does leakage change under multi-hop graph expansion and under a full backend–retrieval–guard–response hybrid pipeline?
RQ9. How robust are the results to provenance errors, stronger response generators, tighter utility metrics, and repeated formal privacy releases?

TABLE I
THREAT-TO-METRIC MAPPING USED BY GRAPHMEMSHIELD.

Threat	Metric	Defense
Edge exposure	leaked edges/events	guard budget
Response leakage	leaked terms/edges	bounded sharing
Session linkage	Top-k, MRR	provenance minim.
Temporal inference	pairwise order acc.	timestamp strip
Edge presence	released/synthetic bits	fixed-univ. RR

TABLE II
CROSS-SESSION LEAKAGE (RQ1, RQ2). STRICT GUARD USES $(\alpha{=}0, b{=}0, \mathcal{B})$.

System	Dataset	Cond.	Leak.	Red.
Synthetic sim.	multisession	baseline	3	–
Synthetic sim.	multisession	strict_guard	0	1.00
PG-Docker	seed	baseline	5	–
PG-Docker	seed	strict_guard	0	1.00
PG-Docker	sample	baseline	48	–
PG-Docker	sample	strict_guard	0	1.00

B. Experimental Setup

We instantiate $\mathcal{A}_g$ with a fixed query bank derived from each domain's seed entities (e.g., *clinic*, *arrhythmia*, *diagnosed*, *gym*, *visited*). Unless a multi-hop experiment explicitly varies the radius, retrievals use $k{=}1$. Strict guard runs use the policy $(\alpha{=}0, b{=}0, \mathcal{B}{=}\{\textit{secret}, \textit{medical}, \textit{financial}\})$. Budgeted runs vary $b \in \{0, 1, 2, 5\}$ and include $b{=}10$ for the enterprise utility curve. Edge-admission runs sweep $\varepsilon \in \{0.1, 1.0, 3.0\}$ and are evaluated over 30 independent trials to compute means and standard deviations.

C. Evaluation Thesis

The evaluation is organised around three claims. First, dynamic KG retrieval creates measurable cross-session leakage in both toy and public-format workloads. Second, leakage is not exhausted by retrieved edge counts: final responses, semantic paraphrases, session fingerprints, and temporal metadata expose additional surfaces. Third, bounded sharing is a practical middle point between unusable strict isolation and unsafe global sharing, but it relies on correct provenance and sensitivity labels.

D. RQ1 & RQ2: Cross-Session Exposure

Table II reports the headline numbers. On the synthetic graph the baseline retrieves 3 unique victim-owned edges (3 leakage events across 3 probe queries, leakage rate 0.0323); on the Docker-backed batch graph the baseline exposes *all* 48 victim-owned edges across 48 queries. Strict isolation reduces both to zero, yielding a leakage reduction of 1.0 in both regimes. The visualisation in Figure 2 makes the gap explicit.

1) Query-Budget Curve: Figure 3 sweeps the per-session query budget on the Docker-backed sample dataset. A single entity-anchored query per session (budget $b{=}1$) already exposes the full set of 48 victim-owned edges; increasing the budget multiplies the *event* count (re-retrievals across queries)

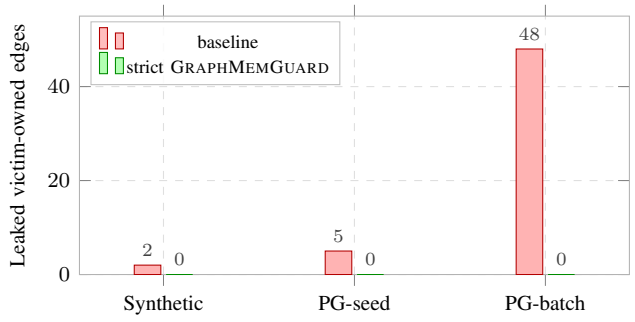

Fig. 2. Leaked victim-owned edges before and after applying GRAPHMEMGUARD (strict). Strict provenance/session isolation eliminates measured leakage on both datasets.

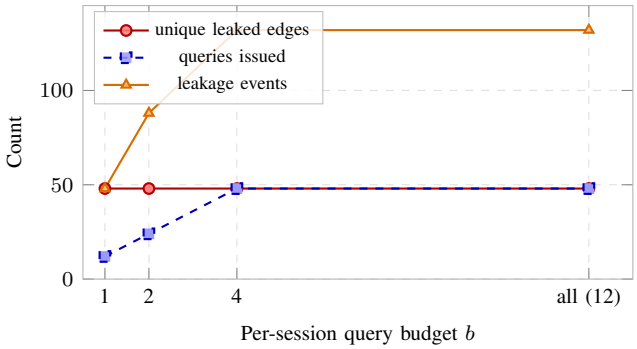

Fig. 3. Query-budget curve on the Docker-backed sample dataset. Unique leaked edges saturate at b=1; leakage events scale linearly with retrieval count.

without raising the unique-edge count. This is a useful auditing signal: it shows that adversarial *coverage* is determined by the schema of the exposed entity surface, not by the number of probes, and that single-shot defenses are sufficient to bound exposure under our adjacency definition.

E. RQ3: Session Linkage

Table III shows that on the Docker-backed sample dataset SESSIONGRAPHLINK achieves Top-1 accuracy of 0.33 from *structure only* (relation, sensitivity, motif and degree features), with all correct matches falling within the Top-3 (1.00 Top-3 accuracy). Adding semantic labels raises MRR from 0.61 to 0.67 but does not change Top-1 in this small regime. Figure 4 visualises the comparison.

F. RQ4: Temporal Path Exposure

On the Docker-backed sample dataset TEMPORALPATHINFER achieves prefix ordering accuracy 0.50, pairwise ordering accuracy 1.00, and edge precision/recall/F1 of 0.49/1.00/0.66 across 12 sessions. This empirically confirms that exposing `created_at` metadata in the retrieval payload is sufficient to recover the relative write-order of every reachable edge; the recall of 1.00 follows directly from the strong adjacency between query terms and seed entities.

G. RQ5: Edge-Admission Sweep

Figure 5 reports the synthetic edge-admission sweep for $\varepsilon \in \{0.1, 1.0, 3.0\}$. The keep-probability rises from 0.525 to 0.953

TABLE III
SESSION-LINKING RESULTS ON DOCKER-BACKED SAMPLE DATASET.

Feature condition	Top-1	Top-3	MRR
structure-only	0.333	1.000	0.611
structure + semantic	0.333	1.000	0.667

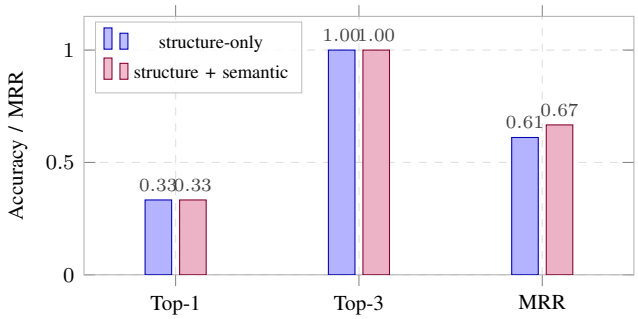

Fig. 4. SESSIONGRAPHLINK accuracy. Adding semantic tokens improves MRR but not Top-1 in this regime.

across the sweep. Across 30 trials, mean victim leakage rises from 1.00 to 2.73 unique edges, with standard deviation falling from 0.98 to 0.64 as retention becomes nearly deterministic. The measured non-sensitive utility retention is 1.00 for all three settings in this synthetic workload. We emphasise that the seeded mechanism is a controlled proxy: the curve stress-tests the audit harness and should not be read as a guarantee of edge differential privacy.

H. Bounded-Sharing Mode

The budgeted policy (α=1, $\mathcal{B}$={*medical*}) is exercised on the synthetic graph against the non-medical Bob session. Increasing the per-pair budget b from 0 to 2 admits 0, 1, and 2 cross-session edges respectively, with medical edges remaining blocked. This illustrates that GRAPHMEMGUARD can interpolate between strict isolation and bounded sharing, providing operators with a direct lever for the privacy–utility tradeoff at retrieval time.

On the larger enterprise/health/finance benchmark, bounded sharing is the primary defensive setting rather than an appendix result. Strict isolation proves that provenance filtering can sever the cross-session boundary, but many graph-memory applications need some collaborative recall. With sensitive labels blocked, increasing b from 0 to 10 raises non-sensitive utility retention from 0.000 to 0.614, while unique victim-edge leakage is capped at 0, 1, 2, 3, and 3 edges for $b \in \{0, 1, 2, 5, 10\}$. This curve is the central defense result: it gives operators a visible control surface for deciding how much cross-session graph utility is worth the residual exposure.

I. Black-Box Response Leakage

We add a deterministic response-level scorer that only observes final text generated from retrieved graph context. We compare a literal template generator, a local abstractive generator, and an optional OpenAI-backed generator when an API key is configured. On the enterprise benchmark, the

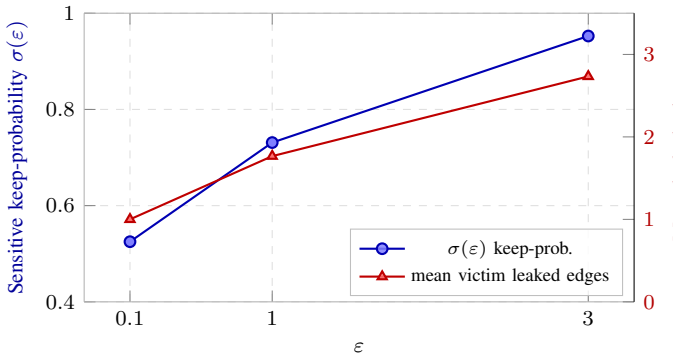

Fig. 5. RANDOMIZEDEDGEADMISSION sweep on the synthetic multisession graph. Keep-probability follows $\sigma(\varepsilon)$; mean downstream victim leakage rises with the one-sided retention probability.

unguarded template response to an attacker query reveals 12 victim-owned edges and 8 sensitive victim terms. Strictly bounded sharing with b=0 reduces both counts to zero. At b=5, the response still reveals 3 victim-owned edges and 2 sensitive terms, demonstrating why edge-level retrieval metrics must be complemented with black-box response scoring. In the local artifact run, the OpenAI-backed condition is recorded as skipped when no API key is present rather than silently replaced by a template.

J. Backend and Stronger Attack Baselines

The persistent property-graph roundtrip preserves all 162 nodes, 2,304 edges, and 192 sessions; running CROSSSESSIONPROBE on the reloaded graph finds 9 leaked victim edges under the unguarded policy. The optional Kuzu backend runs the same experiment when its Python package is available; otherwise the result table records the missing dependency. For session linkage, we evaluate cosine feature matching, 1-WL features, a graph-edit-distance proxy, and hashed embeddings on 24 users. Structure-only variants remain weak in this hard cross-domain setting, while the embedding variant reaches Top-1/Top-3/MRR of 0.042/0.125/0.166. The result is intentionally reported as a baseline rather than an attack ceiling: it shows that cross-domain session linkage is harder than same-domain linkage and motivates learned graph encoders in future work. For timestamp-hidden temporal inference, a relation-priority/path-continuity heuristic achieves pairwise ordering accuracy between 0.606 and 0.716 across the four enterprise query families.

K. Adaptive Probing and Multi-Seed Confidence Intervals

We add ADAPTIVECROSSSESSIONPROBE, which starts from a small seed query set, observes returned entity and relation tokens, and uses them as follow-up probes until a query budget is exhausted. This models a stronger gray-box adversary than a fixed query bank. We also repeat the bounded-sharing experiment over ten independently generated enterprise benchmark seeds and report means with 95% confidence intervals for leakage and utility retention. These intervals separate policy effects from a single graph generator draw.

L. External MultiWOZ and Communication Graph Runs

The MultiWOZ 2.1 external run confirms that the leakage pattern is not limited to generated enterprise traces. With the default 1,000 dialogue cap, the graph contains 36,408 edges. The unguarded probe leaks 67 victim-owned edges across 536 leakage events. Bounded sharing reduces leaked edges to 0, 1, 2, and 5 for $b \in \{0, 1, 2, 5\}$ respectively. The response-level scorer observes 67 leaked victim edges in the unguarded template response; guarded MultiWOZ response scores are reported as stress-test indicators because common slot values can create conservative black-box matches across unrelated dialogues.

The Enron-compatible communication graph evaluates a different communication data shape: sender–recipient–subject interactions. In the default artifact fixture, the graph contains 447 edges. A 2-hop unguarded communication probe leaks 3 victim-session edges across 24 events. Bounded sharing reduces retrieved leakage to 0, 1, 2, and 2 edges for $b \in \{0, 1, 2, 5\}$. We additionally score final responses with both exact lexical matching and semantic alias matching. The baseline template and evidence-dump generators leak 2 victim edges under both scorers, while the b=0 guarded responses leak zero lexical or semantic victim edges and zero sensitive subject terms.

M. Kuzu Overhead

For the Kuzu backend, writing the 2,304-edge enterprise graph takes 23.14 seconds and reading it back takes 0.056 seconds in the local verification environment. Over 100 retrieval queries, unguarded retrieval averages 2.43 ms/query, strict-guard retrieval averages 1.42 ms/query, and bounded-guard retrieval averages 1.41 ms/query. The guard-enabled cases are faster here because blocked provenance/sensitivity edges shrink the graph-expansion frontier.

N. Multi-Hop and Hybrid-Pipeline Evaluation

We finally evaluate whether the privacy conclusions survive deeper graph expansion and a composed application pipeline. In the enterprise benchmark, moving from 1-hop to 2-hop retrieval increases unguarded victim-edge leakage from 9 to 12 edges and expands unique retrieved edges from 1,956 to the full 2,304-edge graph. A third hop does not increase leakage further because the graph is already saturated. Under bounded sharing with b=5 and blocked sensitive labels, leakage remains fixed at 3 victim edges for 1-, 2-, and 3-hop retrieval, showing that the per-pair budget continues to bind under deeper expansion.

The hybrid pipeline chains a persistent backend reload, adaptive probing, optional guard enforcement, local-abstractive response generation, and black-box response scoring. The unguarded pipeline issues 6 adaptive queries and leaks 12 victim edges and 8 sensitive terms in the final response text. Strict isolation reduces both response leakage counts to zero. The bounded b=5 pipeline also yields zero final-response leakage in this adaptive 2-hop setting because the sensitivity block removes the victim's protected edges before response generation.

O. Robustness, Strong Generators, and Utility

We add a robustness sweep that corrupts sensitivity provenance on the victim session before applying bounded sharing. With no injected sensitivity error, the bounded $b{=}5$ setting leaks 3 victim edges through non-sensitive sharing. Corrupting 10%, 25%, and 50% of the victim's sensitive labels to *normal* raises retrieval leakage to 4, 5, and 5 victim edges and raises final-response leakage under an evidence-dump generator to 6, 9, and 12 victim edges. This makes provenance correctness an explicit assumption of the defense: bounded sharing is robust to multi-hop expansion, but not to systematic sensitivity downgrading.

We also compare a literal template generator, a local abstractive generator, and a worst-case evidence-dump generator under an adaptive 8-query, 2-hop probe. In the unguarded condition all three generators expose 12 victim edges and 8 sensitive terms. Under bounded $b{=}5$, local abstraction exposes zero victim edges, while template and evidence-dump responses expose 3 victim edges and 2 sensitive terms. Finally, we replace coarse utility retention with non-sensitive relevance precision/recall/F1. Bounded sharing keeps precision at 1.0 on the evaluated benign queries, but recall ranges from 0.0 to 0.667, showing that leakage control comes with query-dependent collaborative-utility loss.

P. Semantic Response Leakage

The semantic leakage scorer extends exact edge matching with synonym/alias sets and requires relation, source-side, and target-side evidence before assigning an edge-level leak. This reduces false attribution from shared enterprise terms while still catching paraphrased disclosures such as "private", "sensitive", or "confidential" for secret material. On the enterprise benchmark, semantic edge leakage matches lexical edge leakage in the evidence-dump condition: 12 victim edges unguarded and 3 under bounded $b{=}5$. On the Enron-style communication graph, the scorer distinguishes guarded responses with shared business vocabulary from true sender–subject disclosures, yielding zero semantic edge leakage under $b{=}0$.

Q. Fixed-Universe Privacy Sweep

The fixed-universe randomized-response experiment uses 160 protected candidate edges, including 40 absent candidates. As ε increases from 0.5 to 2.0, released edges increase from $2,280$ to $2,294$ and synthetic absent-edge emissions fall from 16 to 3, reflecting the expected shift from privacy-preserving noise toward faithful release. Unlike the one-sided suppression proxy, this mechanism has a clear fixed-universe privacy statement and a composition cost over released protected bits. We now report the mechanism's probability-level accounting: for $\varepsilon \in \{0.5, 1.0, 2.0\}$, the measured single-release privacy loss equals 0.5, 1.0, and 2.0 respectively, and basic sequential composition gives total privacy cost $k\varepsilon$ for $k \in \{1, 5, 10\}$ repeated releases.

R. Full-Graph Release Privacy

We further account for the difference between protected-edge release privacy and full-graph release privacy. If only sensitive candidate edges are randomized and non-sensitive edges are released verbatim, the guarantee scope is "protected candidate edges only" and the release is not a full-graph DP mechanism. In the enterprise graph, this scoped setting covers $1,152$ protected edges while releasing $1,152$ unprotected edges outside the guarantee. A true full-graph release would require every one of the $2,304$ candidate edge bits to be included in the randomized-response universe. Under basic composition, the full-graph candidate-universe cost is therefore $2,304\varepsilon$ for one release, or $11,520$ when $\varepsilon{=}1$ over five repeated releases. We report these values explicitly to prevent the fixed-universe result from being read as unqualified privacy for arbitrary full graph publication.

VII. Discussion, Scope, and Ethics

A. What the Numbers Mean

The results support a simple interpretation: the privacy boundary of a graph-backed memory is the retrieval policy, not the raw text store. When retrieval is global, entity overlap and k-hop expansion carry victim-owned edges into unrelated sessions and then into final responses. When retrieval is provenance-aware, strict isolation removes this channel, and bounded sharing turns it into an explicit budgeted risk. The enterprise multi-hop and hybrid-pipeline results are the clearest evidence: unguarded retrieval reaches 12 victim edges and leaks them in final text, while the guarded pipeline blocks the protected response leakage in the tested setting. The MultiWOZ and Enron-style runs show that the same audit surface appears in both task-oriented dialogue graphs and communication graphs.

The most important caveat is equally clear. GraphMemGuard can only enforce the policy it sees: if sensitivity provenance is systematically downgraded, response leakage rises. Thus the defense is best understood as a provenance-aware control plane for graph memory, not as an end-to-end guarantee against extraction, bad labeling, or all forms of graph publication.

B. Scope and Limitations

The current artifact is intentionally framed as a benchmark and defensive baseline, not as a fully formal mechanism. We emphasise: (i) the Docker-backed dataset is small and de-identified, while the larger enterprise/health/finance workload is deterministic; the MultiWOZ run improves public-corpus coverage and the Enron-compatible runner covers communication-graph structure, but additional approved PersonaChat, full Enron maildir, or institutional traces remain future work; (ii) the Docker integration exercises the read-path through the Cloud API but not the full `validate-and-prove` pipeline of the underlying PrivacyGuard mock; (iii) SessionGraphLink is heuristic even with the current 1-WL, graph-edit proxy, and hashed-embedding baselines, and learned graph encoders remain future work;

(iv) TEMPORALPATHINFER now includes a timestamp-hidden heuristic but still lacks a learned response-likelihood attacker; (v) RANDOMIZEDEDGEADMISSION remains a non-DP one-sided proxy, and formal privacy claims should use the separate fixed-universe randomized-response mechanism with explicit composition accounting. We stress these limits because the strict-guard reduction of 1.0 would otherwise overstate the framework's contribution.

C. Ethical Considerations

All datasets used in this paper are synthetic or de-identified; no real users or production systems were probed. The framework is released for defensive auditing of graph-backed application memories. Operators integrating GraphMemShield should publish their chosen $(\alpha, b, \mathcal{B})$ policy and document the corresponding upper bound on cross-session exposure. If real enterprise mail or dialogue traces are used, the artifact should be run only on approved, access-controlled corpora and reported in aggregate form.

VIII. CONCLUSION

We presented GraphMemShield, a reproducible auditing and mitigation framework for cross-session privacy leakage in dynamic KG memories of graph-backed applications. The framework formalises edge-event adjacency, provides attack modules for cross-session exposure, structural linkage, temporal inference, adaptive probing, and black-box response leakage, and evaluates retrieval-time bounded sharing together with write-time admission and fixed-universe randomized response. Across synthetic, Docker-backed, enterprise-style, public MultiWOZ, and Enron-style communication workloads, unguarded graph retrieval exposes victim-owned edges, while provenance-aware bounded sharing gives operators an explicit privacy–utility control surface. The strongest results are the multi-hop and hybrid-pipeline runs: unguarded enterprise retrieval reaches 12 victim edges, whereas strict and bounded guarded pipelines remove final-response leakage in the tested setting. Robustness experiments show the remaining assumption clearly: the defense depends on correct sensitivity provenance, and systematic label downgrading re-opens leakage. The open-source artifact packages these attacks, defenses, backends, public-format ingestion paths, and privacy-accounting checks as a TIFS-ready benchmark for future graph-memory privacy mechanisms.

REFERENCES

[1] N. Carlini *et al.*, "Extracting training data from large language models," in *Proc. USENIX Security*, 2021.
[2] F. Mireshghallah, A. Goyal, A. Uniyal, T. Berg-Kirkpatrick, and R. Shokri, "Memorisation in fine-tuned dialogue models," in *Proc. EMNLP*, 2023.
[3] W. Zeng, K. Liu, R. Lin, and B. Hooi, "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," *arXiv:2402.16893*, 2024.
[4] Y. Qi *et al.*, "Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems," *arXiv:2402.17840*, 2024.
[5] J. Wang, Z. Lin, T. Xu, and Q. Liu, "Exposing privacy risks in graph retrieval-augmented generation," *arXiv:2503.05923*, 2025.
[6] X. He, J. Jia, M. Backes, N. Z. Gong, and Y. Zhang, "Stealing links from graph neural networks," in *Proc. USENIX Security*, 2021.
[7] B. Wu, X. Yang, S. Pan, and X. Yuan, "Model inversion attacks against graph neural networks," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, 2022.
[8] I. E. Olatunji *et al.*, "Membership inference attacks on graph neural networks," in *Proc. ACSAC*, 2023.
[9] V. Duddu, A. Boutet, and V. Shejwalkar, "Quantifying privacy leakage in graph embedding," in *Proc. MobiQuitous*, 2020.
[10] M. Hay, C. Li, G. Miklau, and D. Jensen, "Accurate estimation of the degree distribution of private networks," in *Proc. IEEE ICDM*, 2009.
[11] Z. Jorgensen, T. Yu, and G. Cormode, "Publishing attributed social graphs with formal privacy guarantees," in *Proc. ACM SIGMOD*, 2016.
[12] V. Karwa, S. Raskhodnikova, A. Smith, and G. Yaroslavtsev, "Private analysis of graph structure," *ACM Trans. Database Systems*, vol. 39, 2014.
[13] A. Daigavane *et al.*, "Node-level differentially private graph neural networks," in *Proc. ICLR Workshop*, 2021.
[14] S. Sajadmanesh and D. Gatica-Perez, "GAP: Differentially private graph neural networks with aggregation perturbation," in *Proc. USENIX Security*, 2023.
[15] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE S&P*, 2017.
[16] A. Salem *et al.*, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Proc. NDSS*, 2019.
[17] M. Backes *et al.*, "Walk2friends: Inferring social links from mobility profiles," in *Proc. ACM CCS*, 2017.
[18] T. Pasquier *et al.*, "Practical whole-system provenance capture," in *Proc. ACM SoCC*, 2017.